

INSuRE Fall 2024
Final Report - 12/6/2024

Cryptographic Protocol Analysis and Verification

Investigators:

The University of Texas at Dallas Team 1

Nathaniel Simmons - Student Researcher - nis190000@utdallas.edu
Zarish Mehboob - Student Researcher - zxm200007@utdallas.edu
David Leung - Student Researcher - dal190004@utdallas.edu
Arturo Yundt-Pacheco - Student Researcher - aay200001@utdallas.edu
Brian Ricks - Faculty Support - absolutefunk@utdallas.edu

Problem Mentor:

Ed Ziegler - National Security Agency - evziegl@uwe.nsa.gov

Table of Contents

1.0 Executive Summary.....	2
2.0 Introduction.....	2
2.1 Top Level Research and Project Description.....	2
2.2 Purpose Statement.....	2
2.3 Motivation.....	2
3.0 Literature Review Summary.....	3
4.0 Methods and Procedures.....	3
4.1 Characterization of Methods Used.....	3
4.2 Procedures / Plans.....	4
4.3 Schedule.....	4
4.4 Deliverables.....	5
4.5 Limitations and Delimitations.....	5
5.0 Findings.....	6
5.1 Findings Overview.....	6
5.2 Detailed Findings.....	6
5.2.1 Research Tools.....	12
6.0 Issues and Challenges.....	12
7.0 Future Work.....	12
8.0 Conclusions and Recommendations.....	12
8.1 Conclusions.....	12
8.2 Recommendations.....	13
9.0 Bibliography.....	13
10.0 Team Biographies / Tasking.....	13
Appendix A: Acronyms and Terms.....	14
A.1 Acronyms.....	14
A.2 Terms.....	15
Appendix B: CPSA Code.....	15
B.1 Pake0 Implementation.....	15
B.2 Pake1 Implementation.....	16
B.3 SRP3 Implementation.....	17
B.4 BACNET/SC two client implementation.....	19

1.0 Executive Summary

The correctness of cryptographic protocols is growing increasingly important, and to have high assurance of their correctness, they should be formally verified. For this project, we formally analyze and verify the correctness of the BACnet protocol in regards to security.

2.0 Introduction

2.1 Top Level Research and Project Description

The goal of the project is to analyze and verify the correctness of a cryptographic protocol in regards to the security properties it intends to provide. We will also potentially propose ways to improve its security and verify the proposed improvements if any security flaws are found.

2.2 Purpose Statement

The overall goal is to have a report of the results of the verification of a chosen cryptographic protocol and potentially a proposal of improvements to the protocol. Formal verification techniques/tools will be used to perform the work. Each security property that the chosen protocol intends to provide should be verified to be either satisfied or not satisfied. If possible improvements are found, the protocol with the possible improvements can also be verified. Accomplishment of the goal can be measured in terms of how many security properties are successfully verified as either being satisfied or not satisfied. If a security property is unable to be validated, the cause(s) can be further researched to determine if any changes can be made that would allow the property to be validated. If improvements are proposed, then their impact on the protocol's security may also be evaluated. If vulnerabilities are discovered that we cannot immediately suggest a remediation for, then that will be an area of future work.

2.3 Motivation

The correctness of cryptographic protocols is growing increasingly important as we become more reliant on using computers for the transfer of confidential/sensitive information. Cryptographic protocols may not always provide the security guarantees they were designed to provide or only provide them under certain assumptions, and some of those assumptions may no longer be realistic. As such, analyzing and verifying the correctness of these protocols grows in importance as well. By analyzing/verifying

cryptographic protocols, flaws can be identified. These flaws can be further researched, and that can lead to the development of more secure protocols, which can then provide greater assurance for the protection of confidential information.

3.0 Literature Review Summary

Building Automation and Control Network (BACnet) was released in 1995 and has seen widespread use since. BACnet facilitates communications between critical building devices such as heating, ventilation, lighting and other building management systems. BACnet does this through a client-server model, allowing devices to function as either servers or clients, allowing them to request and receive data. Throughout this BACnet defines how devices communicate, what data they exchange, and how they interpret each other's data.

BACnet's effectiveness in managing building systems has seen the protocol capturing as much as 60 percent of the market share. However, BACnet was released before cybersecurity was of much concern in the field of building automation. As such, it was not designed with security in mind, leading to several critical security weaknesses such as unencrypted communication and weak authentication.

BACnet Secure Connect (BACnet/SC) has since been released as a solution to many of BACnet's inherent security flaws. BACnet S/C implements an encrypted datalink layer addition to the standard BACnet protocol, adding encryption, authentication, and data integrity checks in line with the industry standards for IP network infrastructures. However, while it is widely considered to be secure, BACnet/SC has had no formal verification or analysis that proves it to be formally secure. This presents the possibility of unrealized weaknesses or vulnerabilities that have not been identified through a formal security verification.

4.0 Methods and Procedures

4.1 Characterization of Methods Used

Formal analysis and verification was conducted utilizing the Cryptographic Protocol Shapes Analyzer (CPSA), a tool developed by Mitre. To analyze a protocol using CPSA, the protocol must first be modeled/specified using CPSA code. Next, the desired security properties are also specified. After the protocol and security properties are specified, CPSA can then be run to explore all possible protocol executions to determine if the security properties are satisfied or not.

4.2 Plans

1. Receive CPSA training
2. Research BACnet/SC
3. Model BACnet/SC in CPSA
4. Analyze/verify the protocol
5. Document findings

4.3 Schedule

Below is the original project schedule

9/2/24 - 9/8/24	Submit Proposal
9/9/24 - 9/22/24	Start CPSA training and research BACnet/SC protocol
9/23/24 - 10/6/24	Analyze security implementation of protocol
10/7/24 - 10/20/24	Analyze any vulnerabilities discovered
10/21/24 - 10/27/24	Write midterm report and presentation
10/28/24 - 11/10/24	Midterm Presentation & Deliverables (approx.)
11/11/24 - 11/17/24	Generate vulnerability remediations if applicable
11/18/24 - 11/24/24	Verify remediations if applicable
11/25/24 - 12/1/24	Write final report and presentation
12/2/24 - 12/8/24	Final Presentation & Deliverables

Our CPSA training period was extended until 10/17, and implementation of the BACnet protocol in CPSA took place after the midterm presentations. Below is a simplified schedule of work completed.

9/2 - 9/8	Prepare and submit proposal
9/9 - 10/17	CPSA training
10/21 - 10/25	Prepare Midterm report and presentation
10/28 - 11/1	Literary review of BACNet paper
11/4 - 11/22	Implement BACNet/SC in CPSA

12/2 - 12/6	Prepare final report and presentation
-------------	---------------------------------------

4.4 Deliverables

Completion of CPSA training: 10/17

Our team successfully completed training on the CPSA tool with Enis Golaszewski.

Midterm Report no later than: 10/25

This report provided an update on the team's progress, detailed the contributions made thus far, and set out any issues and challenges found.

Final Report no later than: Dec 6

The final report provides details regarding the chosen cryptographic protocol, documents what was accomplished, and presents the finalized findings/results from the analysis and verification of the cryptographic protocol. If there are any suggestions for improvement of the protocol, these would also be documented along with the results from verifying the protocol.

CPSA code no later than: Dec 6

Any and all CPSA code for modeling BACnet/SC should be presentable along with the final presentation and report.

4.5 Limitations and Delimitations

We incorporated several delimitations that we felt were necessary to conduct our analysis. The main security aspect of BACnet/SC's cryptographic communication is transport layer security (TLS). That is why we prioritized analyzing how TLS ensures secure communication instead of looking at broader operational or environmental factors. To conduct our analysis we used CPSA since it is well suited for analyzing network communication logic. CPSA is able to consider the attacker's perspective which is vital in verifying the protocol's logic against attempts to compromise it. Additionally, CPSA is designed to abstract low-level architectural details allowing us to concentrate on the core cryptographic communication in BACnet/SC. This aspect ensured the focus of our analysis remained on the logical security of BACnet/SC. Another delimitation was not accounting for real-world network issues. Examples of such network issues include latency, packet loss, hardware-specific performance challenges, etc. Although these are practical concerns that can impact BACnet/SC implementations, we felt they fell outside the scope of our analysis.

A limitation we faced was not being able to account for misconfiguration errors that can be a result of human error, such as mismanaged certificates or weak passwords. These are common errors in real-world scenarios. However, addressing them would require a separate methodology beyond the scope of our logical protocol analysis.

5.0 Findings

5.1 Findings Overview

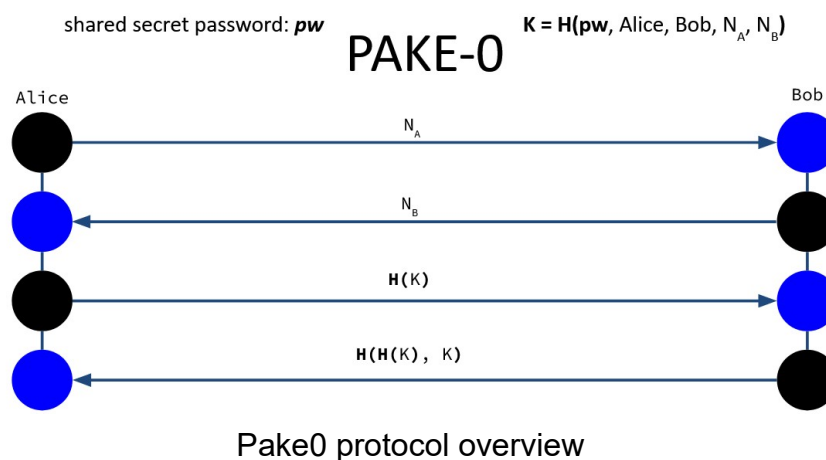
From our formal verification of BACnet/SC, we found that if everything is set-up properly (i.e. certificates, private, public keys, etc.), the protocol is secure.

Additionally, we tested if changing certain attributes would affect the security; however, most of the changes did not impact the results, which was unexpected. See detailed findings for more information on the specific tests.

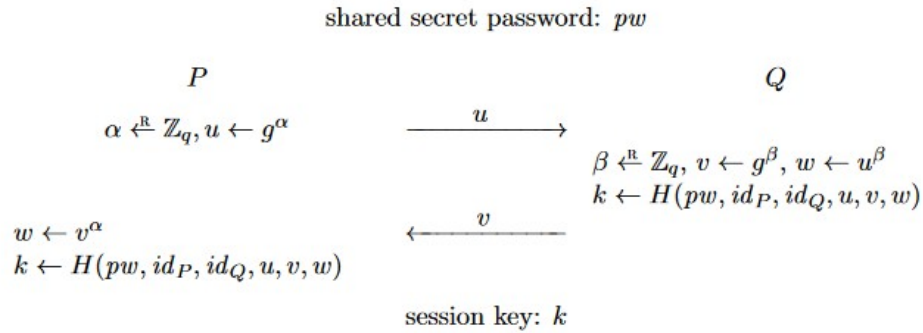
5.2 Detailed Findings

The research team has successfully completed sponsor-mandated training on the CPSA tool.

During this training we first analyzed the Pake0 protocol, which has users share a password, exchange nonces, and agree on a shared key. This is insecure because of the public sharing of the hashed key, and is therefore vulnerable to offline attacks such as dictionary attacks.

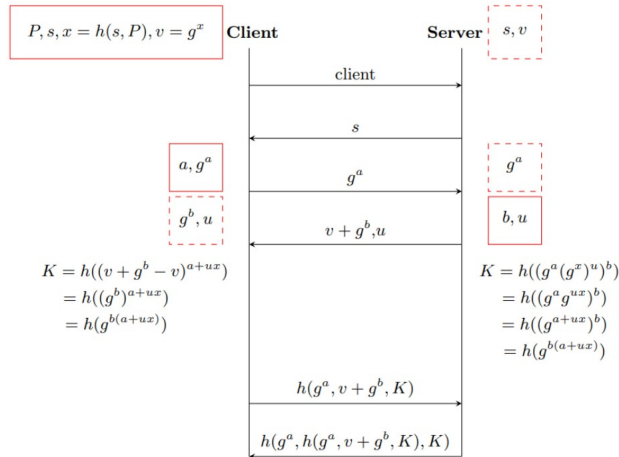


The next protocol we modeled was Pake1. Pake1 also has users maintain a shared password, but only nonces are exchanged, not the entire key hash. Therefore attacks such as a dictionary attack are not possible in the same way as Pake0.



Pake1 protocol overview

Lastly, SRP3 is a password authentication protocol that allows remote authentication. It resists dictionary and man-in-the-middle attacks. It also uses asymmetric key cryptography.



SRP3 protocol overview

See appendix B for CPSA code our team created for these protocols.

Additionally, we have acquired the proprietary specification for BACnet Secure Connect and conducted analyses of the protocol. We identified key parts of BACnet/SC to be modeled to verify its security.

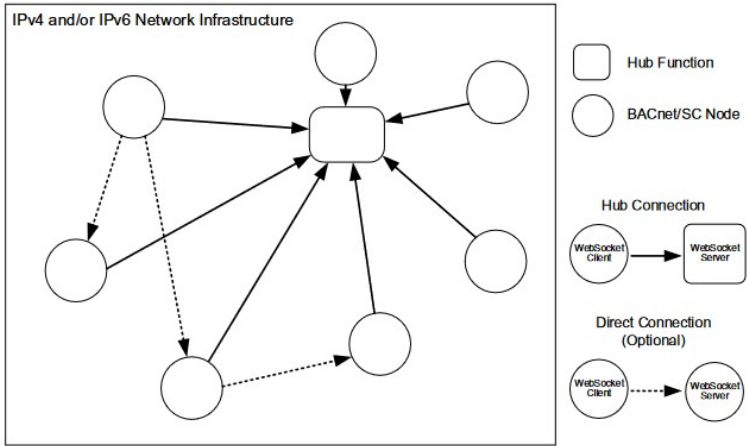


Figure AB-1. BACnet/SC Logical Network Topology

Building Automation and Control Networks, or BACnet, is widely used to manage critical building systems such as HVAC, fire alarms, security, and lighting to name a few. In BACnet, each system is represented by *objects*, which store important properties that describe the system's state or capabilities such as the object type, name, present value, etc. *Services* are used as a method for BACnet devices to communicate with one another. These service requests are sent between devices to perform actions, like adjusting settings or checking system status. This object-service model creates a flexible and efficient way to control and monitor different building systems seamlessly, allowing for integration and automation across various devices.

Object_Name	SPACE TEMP
Object_Type	ANALOG INPUT
Present_Value	72.3
Status_Flags	Normal, InService
High_Limit	78.0
Low_Limit	68.0

The figure above is an example of object properties

An important addendum to the BACnet protocol is BACnet Secure Connect (BACnet/SC) which introduces a major enhancement to the BACnet protocol, adding secure, encrypted communication between devices. This update ensures a protected communication channel, preventing unauthorized access and data tampering, which is essential in building automation systems. In BACnet/SC, the security is applied at the transport layer, where Transport Layer Security (TLS) is used to secure WebSocket connections. WebSockets provide an efficient, real-time, full-duplex communication

channel between devices, and when wrapped with TLS this communication is both persistent and secure. TLS adds three key security features: peer authentication, encryption, and message integrity. To ensure peer authentication TLS verifies device identities using digital certificates, ensuring that only trusted devices can communicate. Encryption between devices prevents any unauthorized party from intercepting or reading sensitive information. Lastly, TLS ensures that messages cannot be tampered with during transmission. If altered, the system detects the modification and rejects the data. This combination of WebSockets and TLS guarantees that data remains private, messages are secure from tampering, and only authenticated devices can participate in the BACnet/SC network.

BACnet Layers								Equivalent OSI Layers
BACnet Application Layer								Application
BACnet Network Layer								Network
ISO 8802-2 (IEEE 802.3) Type 1	MS/TP	PTP	LonTalk	BVLL (Annex J)	BVLL (Annex U)	BZLL	Secure Connect BVLL (Annex YY)	Data Link
ISO 8802-3 (IEEE 802.3)	ARCNET	EIA-485		IPv4	IPv6	Zigbee	WebSocket	Physical

Below is the code for defining an instance of the BACnet/SC client and server. These represent an entity acting as either a TLS websocket server or client respectively. Each role has a number of variables defined, and a trace which represents the communications that occur between strands of each role. We call a macro “mTLS” which stands for mutual TLS authentication within each trace, this macro is a previously existing TLS 1.3 macro provided to our team. Lastly, we make sure to set the secret exponents as uniquely generated because it should not be sent or generated on any other strand

```

(defrole client ;; Role in which the client connects to a server
  (vars
    (c s ca name) ;; c - client u - username, s - server,
    ;; ca - certificate authority
    (cr sr random32) ;; cr - client random, sr - server random for TLS
    (x rndx) ;; client's Diffie-Hellman secret
    (y expt) ;; server's Diffie-Hellman secret
    (spk cpk akey) ;; server's and client's public key's
  )
  (trace
    (mTLS send recv cr sr x y s spk (invk spk) c cpk (invk cpk) ca
      (HandshakeSecret (DHpublic y) x)) ;; establish TLS session with server
    ;;add some message
  )
  (uniq-gen x)
)

(defrole server ;; Role in which the server accepts a client connection
  (vars
    (c s ca name) ;; c - client u - username, s - server,
    ;; ca - certificate authority
    (cr sr random32) ;; cr - client random, sr - server random for TLS
    (y rndx) ;; client's Diffie-Hellman secret
    (x expt) ;; server's Diffie-Hellman secret
    (spk cpk akey) ;; server's and client's public key's
  )
  (trace
    (mTLS recv send cr sr x y s spk (invk spk) c cpk (invk cpk) ca
      (HandshakeSecret (DHpublic x) y)) ;; establish TLS session with client
  )
  (uniq-gen y)
)

```

Next we define the skeletons which contain the strands CPSA will analyze. In the first skeleton we create two clients “clienta” and “clientb” which act as our client strands. In the second paragraph we define the server strands. The client strands are defined with different variables, but the server strands are defined with the same variables. This should result in CPSA generating shapes that have two clients communicating with one server.

```

;;client perspective
(defskeleton bacnet ;;
  (vars (clienta clientb hub ca name) (cra crb sr random32) (y expt) (x rndx) (spk cpka cpkb akey))
  (defstrandmax client (c clienta) (s hub) (ca ca) (cr cra) (spk spk) (cpk cpka))
  (defstrandmax client (c clientb) (s hub) (ca ca) (cr crb) (spk spk) (cpk cpkb))

  (uniq-gen cra crb)
  (non-orig (invk cpka) (invk cpkb) (privk ca) (invk spk) )
)

;; server perspective
(defskeleton bacnet
  (vars (clienta clientb hub ca name) (cra crb sr random32) (y rndx) (x expt) (spk cpka cpkb akey))
  (defstrandmax server (c clienta) (s hub) (ca ca) (sr sr) (spk spk) (cpk cpka))
  (defstrandmax server (c clientb) (s hub) (ca ca) (sr sr) (spk spk) (cpk cpkb))

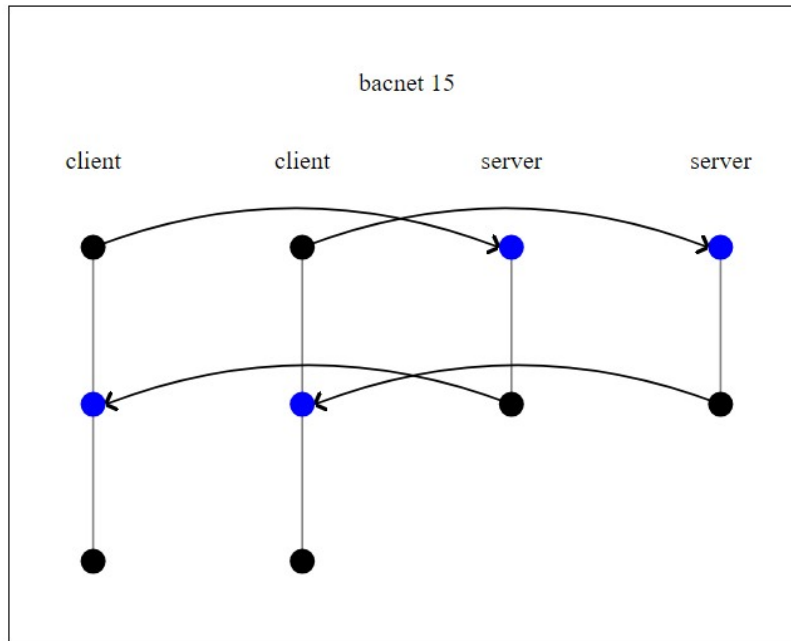
  (uniq-gen sr)
  (non-orig (invk cpka) (invk cpkb) (invk spk) (privk ca))
)

```

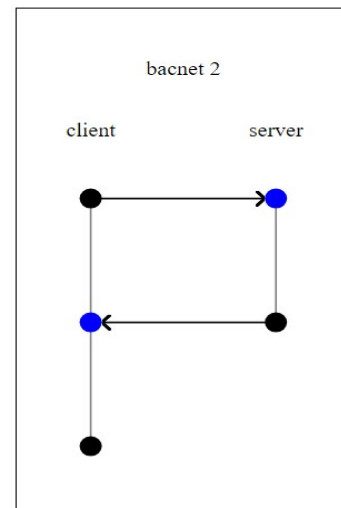
The shapes generated by CPSA are shown below. The shapes match our expectations, on the left is the client perspective with two clients communicating with

server strands. The clients are not aware that they are communicating with the same server strand, so two are shown. On the right is the server strand, which is aware of two clients but since their shapes are identical they are reduced to just one interaction as seen.

Item [15](#), Parent: [0](#).



Item [2](#), Parent: [0](#).



Additional Tests and Results

- Compromised Certificate Authority
 - Removed ca private key from non-orig
 - Shapes Unchanged (Protocol still appears to function securely)
- Clients with same keys
 - 2 clients using the same public and private keys
 - Shapes Unchanged (Protocol still appears to function securely)
- Compromised Client Keys
 - Removed client private key from non-orig
 - Shapes Unchanged (Protocol still appears to function securely)
- Compromised Server Key
 - Removed server private key from non-orig
 - Shapes Changed

- Most shapes show the client(s) not sending any messages (likely due to them not being able to verify the server because of the compromised key)

5.2.1 Research Tools

Cryptographic Protocol Shapes Analyzer (CPSA): Used for formal analysis and verification of the BACnet protocol.

6.0 Issues and Challenges

- Lack of experience with using formal verification techniques/tools
 - This has been a challenge for the team, but with help from the sponsor and the training, we gained sufficient knowledge to model and verify the BACnet protocol
- Protocol Modeling
 - Modeling the protocol was a challenge for a few different reasons
 - BACnet/SC documentation is extensively written and numerous addendums have been made to the protocol.
 - As a result, it took some time to review the documentation and find all the relevant information for modeling the protocol
 - Goals vs. skeletons
 - The main thing we had to model in CPSA for our protocol was mTLS. We were provided with a mTLS macro that had already been created along with some reference files that used the macro. The challenge was that the reference files used goals, which is an alternative to the skeletons we had learned how to create during CPSA training.

7.0 Future Work

Some of the results we found from our additional tests where we tried changing certain attributes to try to see what would affect the protocol's security had results which showed that the protocol appeared to continue functioning correctly even when missing some key features. Due to time limitations, we recommend future research focus in this area.

8.0 Conclusions and Recommendations

8.1 Conclusions

To enhance our understanding of secure authentication protocols and their potential applications to BACnet, we modeled and verified Pake0, Pake1, and SRP3 in CPSA as part of our training. We have learned the core ideas for modeling in CPSA and understanding its complicated output. With this experience we applied our knowledge and skills towards modeling BACnet/SC.

Ultimately, we were able to successfully model, analyze, and verify BACnet/SC using CPSA, and we found that the protocol is secure if everything is set-up properly. We performed some additional tests to try to see if changing some attributes would affect the security and found varying results. Whether due to our inexperience with the tool or lack of knowledge interpreting the results, we are not completely confident in some of the outputs, and thus recommend future work take place in that area. Specifically, we would like to explore the impact of removing certain security attributes to see if it aligns with our prediction of weakening its overall security. By altering these attributes, we can better understand their role in maintaining the system's security strength. For example, our findings indicate that removing the certificates did not affect the security proof as anticipated. Instead, the output remained unchanged, which should not have been the case. Moving forward specifically regarding certificates, we would want to investigate why the removal of them in CPSA did not produce the expected results.

8.2 Recommendations

Users should take care to correctly create and distribute private keys and certificates. The method is not specified in the BACnet/SC protocol, and thus may be a common area for security misconfigurations.

In order to better guide users of the BACnet/SC protocol, we recommend the writers include specific documentation regarding the distribution and configuration of private keys, certificates, etc in the Secure Connect Addendum, rather than leaving it up to the user.

9.0 Bibliography

BACnet A Data Communication Protocol for Building Automation and Control Networks. (2020). ASHRAE, 135–2020.
<https://webstore.ansi.org/standards/ashrae/ansiashrae1352020>

10.0 Team Biographies / Tasking

David has a Bachelor's in Computer Science and is working towards a Master's in Computer Science Cybersecurity and has worked in IT Security at Texas Instruments for 2 internships. One of his internship projects dealt with automating password changes, which couldn't have been done securely without cryptography and encryption. He has also taken a cryptography course and has also learned about/applied cryptography concepts as it relates to networks in a network security course.

Zarish recently graduated with a Bachelor's in Computer Science and is currently studying for her Master's in Computer Science with a concentration in cybersecurity. She has had two internships with government organizations. During her most recent internship she analyzed OT protocols and wrote descriptive reports on identified trends. She has a passion for learning about different networking protocols, how they function, and what their vulnerabilities might be. She attended the DOE CIO conference that exposed her to current cybersecurity trends in critical infrastructure that she is determined to incorporate in future projects.

Arturo Yundt-Pacheco is a fourth-year Computer Science undergraduate student completing his final semester for a bachelors at the University of Texas at Dallas, alongside a certificate in Entrepreneurship and Innovation. Throughout this time he has taken classes and worked on projects utilizing several programming languages, both high and low level. He is also currently taking courses to learn about networking protocols and standards. Outside of classes he has worked to independently increase his cybersecurity knowledge, particularly in topics relevant to incident response such as memory forensics. Outside of the realm of cybersecurity Mr. Yundt-Pacheco has worked on a project with the US Army and an internship with the non-profit NTXIA to develop a keen understanding over operations and organizational strategies.

Nate Simmons recently graduated Cum Laude with his Bachelor's degree in Computer Science, and is continuing to pursue a Master's in Computer Science as a National Science Foundation Scholarship for Service Student. He participates in capture the flag competitions with the computer security group at his university where they are in the top 11% in the US open category. Last summer he completed an internship at Lawrence Livermore National Laboratory. There he focused on finding and demonstrating vulnerabilities in programmable logic controllers. Additionally, he has completed the CompTIA Security+ certification and plans to pursue more professional certifications in the future.

During the project, all team members contributed in different ways throughout the semester. While all team members attended numerous meetings with sponsors and during the training, Arturo and Nate spoke up most often. Zarish was already somewhat familiar with BACnet and took the lead, informing other team members about the protocol. When it came time to write papers and start presentations, Nate created and organized much of the content and David, Zarish, and Arturo helped write the content. Everyone participated in the presentations, and in general the group has minimal issues collaborating, communicating, and working together.

Appendix A: Acronyms and Terms

A.1 Acronyms

ANSI - American National Standards Institute

ASHRAE - The American Society of Heating, Refrigerating and Air-Conditioning Engineers

BACnet - Building Automation and Control Network

BACnet/SC - Building Automation and Control Network Secure Connect

CPSA - Cryptographic Protocol Shapes Analyzer

HVAC&R - Heating, Ventilation, Air Conditioning and Refrigeration

TLS - Transport Layer Security

A.2 Terms

ANSI - A standards institute facilitates the development of consensus standards in the US.

ASHRAE - A professional organization focused on HVAC&R and develop standards related to the field. The paper on BACNET was published in their journal in joint with ANSI.

BACnet - An open, standardized protocol used for communication between building automation and control systems.

BACnet/SC - A security protocol built on top of the BACnet communication protocol.

CPSA - A tool developed by MITRE used to analyze and evaluate the security properties of cryptographic protocols.

HVAC&R - A broad field encompassing the design, installation, operation, and maintenance of systems that control the temperature, humidity, and air quality in buildings.

TLS - A cryptographic protocol that provides security for communication over a computer network.

Appendix B: CPSA Code

B.1 Pake0 Implementation

```

(defprotocol pake0 basic
  (defrole alice
    (vars (a b name) (noncea nonceb data) (pwd skey))
    (trace
      (send noncea)
      (recv nonceb)
      (send (hash pwd noncea nonceb a b))
      (recv (hash (hash pwd noncea nonceb a b) pwd noncea nonceb a b))
    )
  )
  (defrole bob
    (vars (a b name) (noncea nonceb data) (pwd skey))
    (trace
      (recv noncea)
      (send nonceb)
      (recv (hash pwd noncea nonceb a b))
      (send (hash (hash pwd noncea nonceb a b) pwd noncea nonceb a b))
    )
  )
)

(defskeleton pake0
  (vars (a b name) (noncea nonceb data) (pwd skey))
  (defstrandmax alice (a a) (b b) (noncea noncea) (nonceb nonceb) (pwd pwd))
  (uniq-orig noncea nonceb)
  (pen-non-orig pwd)
)

(defskeleton pake0
  (vars (a b name) (noncea nonceb data) (pwd skey))
  (defstrandmax bob (a a) (b b) (noncea noncea) (nonceb nonceb) (pwd pwd))
  (uniq-orig noncea nonceb)
  (pen-non-orig pwd)
)

(defskeleton pake0
  (vars (a b name) (noncea nonceb data) (pwd skey))
  (defstrandmax alice (a a) (b b) (noncea noncea) (nonceb nonceb) (pwd pwd))

```

```

    (deflistener pwd)
    (uniq-orig noncea nonceb)
    (pen-non-orig pwd)
  )
  (defskelton pake0
    (vars (a b name) (noncea nonceb data) (pwd skey))
    (defstrandmax bob (a a) (b b) (noncea noncea) (nonceb nonceb) (pwd pwd))
    (deflistener pwd)
    (uniq-orig noncea nonceb)
    (pen-non-orig pwd)
  )

```

B.2 Pake1 Implementation

```

(defprotocol pake1 basic
  (defrole alice
    (vars (a b name) (na nb rndx) (pwd skey))
    (trace
      (send (exp (gen) na));; u
      (recv (exp (gen) nb));; v
      ;; send H(k)
      (send (hash (hash pwd a b (exp (gen) na) (exp (gen) nb) (exp (exp (gen) na)
nb)))) ;; h(pwd a b u v w) = k
      ;; send H(H(k) k)
      (recv (hash (hash (hash pwd a b (exp (gen) na) (exp (gen) nb) (exp (exp (gen)
na)nb)))) (hash pwd a b (exp (gen) na) (exp (gen) nb) (exp (exp (gen) na) nb))))
    )
    (uniq-gen na)
  )
  (defrole bob
    (vars (a b name) (na nb rndx) (pwd skey))
    (trace
      (recv (exp (gen) na));; u
      (send (exp (gen) nb));; v
      ;; send H(k)
      (recv (hash (hash pwd a b (exp (gen) na) (exp (gen) nb) (exp (exp (gen) na)
nb)))) ;; h(pwd a b u v w) = k
      ;; send H(H(k) k)
      (send (hash (hash (hash pwd a b (exp (gen) na) (exp (gen) nb) (exp (exp (gen)
na)nb)))) (hash pwd a b (exp (gen) na) (exp (gen) nb) (exp (exp (gen) na) nb))))
    )
  )

```

```

        (uniq-gen nb)
      )
    )
  (defskelton pake1
    (vars (a b name) (na nb rndx) (pwd skey))
    (defstrandmax alice (a a) (b b) (na na) (pwd pwd))
    (pen-non-orig pwd)
  )
  (defskelton pake1
    (vars (a b name) (na nb rndx) (pwd skey))
    (defstrandmax bob (a a) (b b) (nb nb) (pwd pwd))
    (pen-non-orig pwd)
  )

```

B.3 SRP3 Implementation

```

(defprotocol srp basic
  (defrole client
    ;; pwd salt x(hash) na nb
    (vars (c name) (a b u x rndx) (salt data) (gx locn))
    (trace
      (load gx "init")
      (stor gx (exp (gen) x))
      (send c)
      (recv salt)
      (send (exp (gen) a))
      (recv (cat (enc (exp (gen) b) (exp (gen) x)) u))
      ;;  $K = (\text{hash}(\text{hash}(\text{exp}(\text{gen}) (\text{mul } a \ b)) (\text{exp}(\text{gen}) (\text{mul } b \ u \ x))))$ 
      ;;  $h(g^a(ab) + g^a(bux)) = h(g^a(ab + bux))$ 
      (send (hash (exp (gen) a) (enc (exp (gen) b) (exp (gen) x)) (hash (hash (exp
(gen) (mul a b)) (exp (gen) (mul b u x))))))
      ;;  $h(g^a \text{ above } K)$ 
      (recv (cat (hash (exp (gen) a) ;;  $g^a$ 
      (hash (exp (gen) a) (enc (exp (gen) b) (exp (gen) x)) (hash (hash (exp (gen) (mul
a b)) (exp (gen) (mul b u x)))))) ;;  $h(g^a(ab) + g^a(bux)) = h(g^a(ab + bux))$ 
      (hash (hash (exp (gen) (mul a b)) (exp (gen) (mul b u x)))))) ;;  $K$ 
    )
    (uniq-gen a x)
  )
  (defrole server
    (vars (c name) (a b u x rndx) (salt data) (gx locn))

```

```

(trace
(load gx (exp (gen) x))
(recv c)
(send salt)
(recv (exp (gen) a))
(send (cat (enc (exp (gen) b) (exp (gen) x)) u))
;; K = (hash (hash (exp (gen) (mul a b)) (exp (gen) (mul b u x))))
;;  $h(g^{ab} + g^{bux}) = h(g^{ab + bux})$ 
(recv (hash (exp (gen) a) (enc (exp (gen) b) (exp (gen) x)) (hash (hash (exp (gen)
(mul a b)) (exp (gen) (mul b u x))))))
;;  $h(g^a \text{ above } K)$ 
(send (cat (hash (exp (gen) a) ;;  $g^a$ 
(hash (exp (gen) a) (enc (exp (gen) b) (exp (gen) x)) (hash (hash (exp (gen) (mul
a b)) (exp (gen) (mul b u x)))))) ;;  $h(g^{ab} + g^{bux}) = h(g^{ab + bux})$ 
(hash (hash (exp (gen) (mul a b)) (exp (gen) (mul b u x)))))) ;; K
)
(uniq-gen b u)
)
)
(defskeleton srp
  (vars (c name) (a b u x rndx) (salt data))
  (defstrandmax client (c c) (a a) (b b) (u u) (x x) (salt salt))
)
(defskeleton srp
  (vars (c name) (a b u x rndx) (salt data))
  (defstrandmax server (c c) (a a) (b b) (u u) (x x) (salt salt))
  (deflistener x)
  (uniq-gen x)
)

```

B.4 BACNET/SC two client implementation

```

;; 11/22/2024
;; This file contains code for CPSA code and was created for the CS 6301 Insure class
at University of Texas at Dallas
;; Specifically we attempt to implement the BACNET/SC protocol for verification
(herald "bacnet"
  (try-old-strands)
)

```

(include "TLS1.3_macros.lisp") ;; 3 message exchange by combining TLS messages.

(include "mtls_rules.lisp") ;; Proven rules to control CPSA analysis.

;; The following macro is a shortcut to create the TLS master secret.

```
(defmacro (MS exponent1 exponent2)
  (MasterSecret (HandshakeSecret (DHpublic exponent1) exponent2)))
```

(defprotocol bacnet basic

(defrole client ;; Role in which the client connects to a server

```
  (vars
    (c s ca name) ;; c - client u - username, s - server,
                  ;; ca - certificate authority
    (cr sr random32) ;; cr - client random, sr - server random for TLS
    (x rndx) ;; client's Diffie-Hellman secret
    (y expt) ;; server's Diffie-Hellman secret
    (spk cpk akey) ;; server's and client's public key's
  )
  (trace
    (mTLS send recv cr sr x y s spk (invk spk) c cpk (invk cpk) ca
      (HandshakeSecret (DHpublic y) x)) ;; establish TLS session with server
    ;;add some message
  )
  (uniq-gen x)
)
```

(defrole server ;; Role in which the server accepts a client connection

```
  (vars
    (c s ca name) ;; c - client u - username, s - server,
                  ;; ca - certificate authority
    (cr sr random32) ;; cr - client random, sr - server random for TLS
    (y rndx) ;; client's Diffie-Hellman secret
    (x expt) ;; server's Diffie-Hellman secret
    (spk cpk akey) ;; server's and client's public key's
  )
  (trace
    (mTLS recv send cr sr x y s spk (invk spk) c cpk (invk cpk) ca
      (HandshakeSecret (DHpublic x) y)) ;; establish TLS session with client
  )
```

```

    )
    (uniq-gen y)
  )
;; Rules to control the analysis

(clients-distinct-from-servers-rules)

(lang
  (random32 atom)
)

;;client perspective
(defskeleton bacnet ;;
  (vars (clienta clientb hub ca name) (cra crb sr random32) (y expt) (x rndx) (spk
cpka cpkb akey))
  (defstrandmax client (c clienta) (s hub) (ca ca) (cr cra) (spk spk) (cpk cpka))
  (defstrandmax client (c clientb) (s hub) (ca ca) (cr crb) (spk spk) (cpk cpkb))

  (uniq-gen cra crb)
  (non-orig (invk cpka) (invk cpkb) (privk ca) (invk spk) )
)

;; server perspective
(defskeleton bacnet
  (vars (clienta clientb hub ca name) (cra crb sr random32) (y rndx) (x expt) (spk
cpka cpkb akey))
  (defstrandmax server (c clienta) (s hub) (ca ca) (sr sr) (spk spk) (cpk cpka))
  (defstrandmax server (c clientb) (s hub) (ca ca) (sr sr) (spk spk) (cpk cpkb))

  (uniq-gen sr)
  (non-orig (invk cpka) (invk cpkb) (invk spk) (privk ca))
)

```